

Escopo

Dashboard, Plantões, Equipe, Relatórios e Configurações.

Base atual

Next.js App Router com componentes React e dados mockados.

Padrão visual

Layout sem bordas arredondadas, fundo escuro e superfícies neutras.

Consumo esperado

Mesma estrutura de objetos para facilitar troca por API.

Objetivo do documento

Este PDF foi criado para orientar o front-end sobre a estrutura atual do sistema Meu Plantão. Ele detalha rotas, componentes, contratos de dados, estados de interface, comportamento do tema escuro, idioma e as regras visuais que precisam ser mantidas ao consumir dados vindos de uma API, backend ou mock local.

Use este guia como contrato funcional do front-end.

1. Visão geral do sistema

A aplicação foi desenhada como um painel administrativo operacional para gestão de plantões, equipe e relatórios. O front-end já está organizado para receber dados locais ou externos sem mudar a experiência visual.

1.1 Estrutura de navegação

Rota	Página	Objetivo
/pages/dashboard	Dashboard	Resumo operacional com filtros, cards, gráfico semanal, lista de atenção e tabela recente.
/pages/plantoes	Plantões	Lista completa de solicitações com ações de aprovar, recusar, visualizar, editar e excluir.
/pages/equipe	Equipe	Gestão de usuários com busca, filtro por perfil, criação, edição e exclusão.
/pages/relatorios	Relatórios	Histórico consolidado com filtros, resumo financeiro e exportação em PDF.
/pages/configuracoes	Configurações	Preferências do sistema com idioma, nome do sistema, e-mail e tema automático.
/pages/login	Login	Entrada inicial do sistema.

1.2 Layout estrutural

- Cada página operacional usa o mesmo shell visual: `Sidebar` fixa à esquerda, plano de fundo com partículas, camada radial escura e conteúdo principal alinhado à direita no desktop.
- O conteúdo principal trabalha com espaçamento de 16 a 32 px, cards sem bordas arredondadas e contraste forte entre texto e superfície.
- A sidebar é a fonte da navegação global. Os itens apontam para as rotas em `/pages/...` e o idioma do texto muda conforme o estado global de linguagem.
- O botão de tema fica no rodapé da sidebar e depende do `ThemeProvider` do `next-themes`. O idioma usa `LanguageProvider` e `localStorage`.

Regra principal de integração

Quando houver troca para API, preserve a forma dos objetos já usados pelo front-end. O ideal é devolver exatamente os mesmos campos dos seeds locais para evitar refatoração de componentes.

1.3 Linguagem e tema

Campo	Descrição
Idioma	O estado global atual trabalha com pt-BR e en. Ao selecionar English, toda a navegação e textos compatíveis podem ser exibidos em inglês.
Tema escuro	O dark mode deve permanecer bem escuro, com superfícies graphite/neutral e sem viés azul forte.
Persistência	Idioma é salvo em localStorage. O tema é controlado pelo provider do `next-themes`.
Toggle de tema	O usuário alterna o modo escuro diretamente na sidebar.

2. Contratos de dados

A camada de componentes foi construída sobre tipos TypeScript claros. Eles servem como contrato ideal para API ou store local.

2.1 Plantões

Campo	Descrição
PlantaoItem.id	Identificador único do plantão. Ex.: PL-2048.
PlantaoItem.unidade	Unidade hospitalar ou setor de atendimento.
PlantaoItem.profissional	Nome do profissional escalado.
PlantaoItem.especialidade	Área clínica relacionada ao plantão.
PlantaoItem.periodo	Faixa de horário exibida na tabela e modal.
PlantaoItem.solicitante	Origem da solicitação ou setor responsável.
PlantaoItem.status	Pendente, Aprovado ou Recusado.
PlantaoItem.observacao	Texto auxiliar para contextualizar a solicitação.

2.2 Equipe

Campo	Descrição
EquipeMember.id	ID de usuário no formato USR-001.
EquipeMember.name	Nome completo do usuário.
EquipeMember.email	E-mail principal.
EquipeMember.role	Admin, Profissional ou Gestor.
EquipeMember.department	Setor ou departamento.
EquipeMember.status	Ativo ou Inativo.
EquipeMember.notes	Observações administrativas e operacionais.

2.3 Relatórios

Campo	Descrição
RelatorioPlantao.id	ID do relatório, normalmente no padrão RPT-001.
RelatorioPlantao.unidade	Unidade de origem do registro.

Campo	Descrição
RelatorioPlantao.profissio nal	Profissional do plantão.
RelatorioPlantao.conclu idoPor	Usuário que concluiu ou conferiu o relatório.
RelatorioPlantao.data	Data exibida na tabela.
RelatorioPlantao.valorG erado	Valor numérico para cálculo de receita.
RelatorioPlantao.status	Completo, Em aberto ou Cancelado.

2.4 Configurações

Campo	Descrição
nomeSistema	Nome exibido como identidade principal do sistema.
emailAdmin	E-mail do administrador.
idioma	Português (Brasil) ou English.
darkMode	Flag booleana para aplicar tema escuro automático.

3. Dashboard

A dashboard resume a operação do dia. Ela é a principal página de visão rápida e deve continuar leve, legível e visualmente estável mesmo quando os dados mudarem.

3.1 Componentes principais

- Header com badge de seção, título e descrição curta.
- Filtro global com campo de busca, período, status e botão de atualização.
- Quatro summary cards: Aprovados, Recusados, Pendentes e Total de plantões.
- Gráfico semanal em barras para leitura rápida de volume.
- Lista de atenção com os itens mais urgentes.
- Tabela de plantões recentes com código, unidade, profissional, período e status.

3.2 Regras dos summary cards

Campo	Descrição
Card Aprovados	Mostra volume liberado. Usa tratamento visual positivo com progress bar em verde.
Card Recusados	Mostra rejeições. Usa tom negativo em rosa/vermelho.
Card Pendentes	Mostra itens que ainda exigem ação. Usa tom de alerta em âmbar.
Card Total de plantões	Card neutro, sem puxar para azul. Esse card foi convertido para família graphite.

3.3 Consumo de dados esperado

- O front-end trabalha bem com objetos agregados vindos de API, por exemplo `{ approved, rejected, pending, total }`.
- O gráfico semanal precisa receber uma coleção com `{ day, value }` para renderização direta.
- A lista de atenção deve receber itens com `title`, `subtitle` e `badge`.
- A tabela recente espera colunas com `id`, `unidade`, `profissional`, `periodo` e `status`.

4. Plantões

Essa é a tela operacional mais importante para aprovar ou recusar plantões. Ela precisa de leitura rápida, ações diretas e modais sem ambiguidades.

4.1 Fluxo funcional

- O usuário pesquisa por código, unidade, profissional, especialidade ou solicitante.
- O filtro por status reduz os itens para Pendente, Aprovado ou Recusado.
- Cada linha da tabela permite visualizar detalhes, editar informações, aprovar, recusar e excluir.
- A ação de aprovar/recusar abre um modal de confirmação antes de alterar o status.
- O modal de edição usa o mesmo item selecionado e grava a versão editada no array de dados.

4.2 Dados que o front-end consome

Campo	Tipo	Uso na interface
id	string	Exibido como referência e usado na chave da linha.
unidade	string	Texto principal de identificação.
profissional	string	Nome exibido em destaque.
especialidade	string	Badge auxiliar e refinamento de busca.
periodo	string	Faixa de horário do plantão.
solicitante	string	Mostra a origem da solicitação.
status	string	Define badge, filtro e ação de aprovação.
observacao	string	Conteúdo do painel lateral de detalhes.

4.3 Estados visuais

- Pendente precisa continuar em âmbar para sinalizar ação necessária.
- Aprovado precisa manter verde, porque é um estado de sucesso operacional.
- Recusado deve continuar rosa/vermelho, sem tendência azul.
- Todos os cards e painéis da tela usam superfícies neutras escuras quando o modo escuro está ativo.

5. Equipe

A página de equipe serve para administrar usuários do sistema. Ela precisa equilibrar filtro rápido, CRUD básico e leitura limpa dos perfis.

5.1 Comportamento da página

- O topo apresenta badge de seção, título e texto de orientação.
- Existe busca por nome, e-mail, departamento, cargo e status.
- Há filtro por perfil com opções Todos, Admin, Profissional e Gestor.
- O botão "Novo usuário" inicia o fluxo de criação com modal.
- O modal é reaproveitado para visualizar, editar e criar usuários.

5.2 Modelo de usuário

Campo	Tipo	Observação
id	string	Identificador do registro.
name	string	Nome do usuário.
email	string	E-mail principal para login ou contato.
role	string	Define permissões e badge.
department	string	Setor de atuação.
status	string	Ativo ou Inativo.
notes	string	Texto livre para contexto administrativo.

5.3 Regras visuais da equipe

- O perfil Admin foi ajustado para tons neutros, porque ele era um dos pontos azuis mais fortes do sistema.
- Os demais perfis podem continuar com tons diferenciados, desde que não reintroduzam azul forte.
- A tabela e o modal devem manter a mesma geometria sem bordas arredondadas.

6. Relatórios

A página de relatórios reúne o histórico consolidado e é o ponto ideal para exportação e leitura financeira. Ela precisa ser mais analítica do que operacional.

6.1 O que a página mostra

- Resumo consolidado com totais de registros, completos, em aberto e receita acumulada.
- Filtro por texto e status.
- Botão de exportação em PDF usando os dados já filtrados.
- Tabela com unidade, profissional, responsável pela conclusão, data, valor e status.

6.2 Consumindo os relatórios

Campo	Tipo	Uso
id	string	Código do relatório.
unidade	string	Unidade do plantão.
profissional	string	Nome do profissional.
concluidoPor	string	Responsável pela conclusão.
data	string	Data textual para exibição rápida.
valorGerado	number	Valor monetário para somatório.

Campo	Tipo	Uso
status	string	Completo, Em aberto ou Cancelado.

6.3 Exportação PDF

- O botão de exportar deve receber exatamente a lista já filtrada.
- Se houver API, o front-end não deve recalcular o estado exportado fora do mesmo filtro aplicado na tela.
- A exportação precisa refletir somente os campos mostrados no relatório, para evitar inconsistência.

7. Configurações

As configurações agora são um formulário de rascunho: o usuário altera os campos localmente e só aplica as mudanças ao clicar em salvar alterações.

7.1 Campos disponíveis

- Nome do sistema.
- E-mail do administrador.
- Idioma com suporte a Português (Brasil) e English.
- Modo escuro automático.

7.2 Comportamento de persistência

- Nada deve ser aplicado instantaneamente ao digitar ou alternar o seletor.
- Salvar alterações é o único ponto que deve disparar mudança de idioma e tema.
- O formulário mantém um estado salvo e um estado rascunho, para o contador de pendências funcionar corretamente.
- O idioma global fica armazenado em localStorage, enquanto o tema é decidido pelo provider do `next-themes`.

7.3 Expectativa para o front-end

Importante

Se a tela de configurações passar a consumir uma API, mantenha o conceito de rascunho e confirmação. O usuário não pode ter o idioma ou o tema alterados antes de confirmar a ação.

8. Regras de consumo para o front-end

Essas são as regras práticas que ajudam a evitar quebra quando os dados vierem de um backend real.

- Preserve os nomes dos campos já usados nas tabelas e modais.
- Mantenha os status como enums fechados. Isso simplifica filtros e badges.
- Evite devolver dados num formato diferente entre telas que representam a mesma entidade.
- Se a API paginar resultados, o componente precisa ser adaptado para paginação sem quebrar o filtro local.
- Se o front-end precisar exibir loading, preserve as mesmas larguras dos cards para evitar layout shift.

8.1 Recomendações de endpoints

Endpoint sugerido	Método	Finalidade
/api/dashboard/summary	GET	Agregados para os cards e indicadores do resumo.
/api/dashboard/weekly-volume	GET	Dados do gráfico semanal.

Endpoint sugerido	Método	Finalidade
/api/plantoes	GET	Lista com busca e filtros.
/api/plantoes/:id/status	PATCH	Aprovar ou recusar um plantão.
/api/plantoes/:id	PATCH/DELETE	Editar ou excluir o plantão.
/api/equipe	GET	Listagem de usuários.
/api/equipe	POST	Criar usuário.
/api/relatorios	GET	Histórico consolidado.
/api/configuracoes	GET/PATCH	Ler e salvar preferências do sistema.

8.2 Contrato de resposta ideal

Formato recomendado

Retorne objetos com chaves claras, tipos estáveis e sempre os mesmos rótulos de status. Isso mantém os componentes atuais funcionando sem retrabalho visual ou estrutural.

9. Checklist final para o front-end

Antes de integrar com dados remotos, este é o checklist que deve ser revisado.

- Os filtros de todas as páginas estão padronizados em superfície neutra escura.
- Os cards da dashboard não usam mais a família azul como destaque principal.
- O fundo geral continua escuro e atmosférico, sem precisar ser modificado para reduzir o azul.
- Os textos e badges devem continuar legíveis no dark mode.
- Os estados de idioma e tema devem continuar separados do salvamento do formulário de configurações.

Resumo

Se o objetivo é consumir uma API real, o melhor caminho é manter exatamente esses contratos e substituir apenas a origem dos dados. Assim o front-end continua previsível, testável e fácil de evoluir.